COMP3242/6242: Deep Learning

Week 7 — Developing, Debugging and Diagnosing Deep Learning Algorithms

Stephen Gould
stephen.gould@anu.edu.au

Australian National University

Semester 1, 2026

Overview

- Numerical Issues
- Algorithmic/Design Issues
- Bugs Can be Hard to Find
- Dataset Issues
- Diagnostics
- Advice for Getting Unstuck
- Tools: Weights & Biases

Simple Arithmetic Quiz

- **Question 1:** What is $255 + 1$?

Simple Arithmetic Quiz

- **Question 1:** What is $255 + 1$?
- **Answer:** 0

```
import numpy as np

x = np.array([0, 1, 41, 255], dtype='uint8')
x += 1
print(x)
```

Simple Arithmetic Quiz

- **Question 1:** What is 255 + 1?
- **Answer:** 0

```
import numpy as np

x = np.array([0, 1, 41, 255], dtype='uint8')
x += 1
print(x)
```

- **Question 2:** What is 16,777,216 + 1?

Simple Arithmetic Quiz

- **Question 1:** What is $255 + 1$?
- **Answer:** 0

```
import numpy as np

x = np.array([0, 1, 41, 255], dtype='uint8')
x += 1
print(x)
```

- **Question 2:** What is $16{,}777{,}216 + 1$?
- **Answer:** 16,777,216

```
import numpy as np

x = np.array([16777216], dtype='float32')
x += 1
print(x)
```

Pixel Manipulation

- image manipulation (e.g, filtering) is susceptible to these types of underflow and overflow

```
1   import numpy as np
2   import cv2
3
4   img = cv2.cvtColor(cv2.imread(FILE_IN), cv2.COLOR_BGR2RGB)
5   img2 = img + 1
6   cv2.imwrite(FILE_OUT, cv2.cvtColor(img2, cv2.COLOR_RGB2BGR))
7
8   import matplotlib.pyplot as plt
9   plt.figure()
10  plt.subplot(2,1,1)
11  plt.imshow(img); plt.axis('off')
12  plt.subplot(2,1,2)
13  plt.imshow(img2); plt.axis('off')
14  plt.show()
```

Numerical Issues

"anything that can go wrong will go wrong"

Numerical Issues

"anything that can go wrong will go wrong"

- numerical calculations on a computer are always subject to errors
- deep learning algorithms are full of numerical calculations
- numerical errors can be due to:
  - limited precision arithmetic
    - we just saw some examples
  - algorithmic limitations (e.g., generating true random numbers)
  - careless implementations
    - we will see some example soon
  - bugs!!!

Numerically Stable Standard Deviation

- empirical standard deviation is defined as

$$\hat{\sigma} = \sqrt{\frac{\sum_{i=1}^{n} (x_i - \mu)^2}{n-1}}$$

  where $\mu = \frac{1}{n} \sum_{i=1}^{n} x_i$
- requires two passes through the data

Numerically Stable Standard Deviation

- empirical standard deviation is defined as

$$\hat{\sigma} = \sqrt{\frac{\sum_{i=1}^{n} (x_i - \mu)^2}{n-1}}$$

  where $\mu = \frac{1}{n} \sum_{i=1}^{n} x_i$
- requires two passes through the data
- a seemingly better approach is to perform equivalent calculation

$$\hat{\sigma} = \sqrt{\frac{n \sum_{i=1}^{n} x_i^2 - \left(\sum_{i=1}^{n} x_i\right)^2}{n(n-1)}}$$

  which only requires one pass
- but what can go wrong with this implementation?

Numerically Stable Softmax

- recall the definition of softmax for a vector $z \in \mathbb{R}^K$ is

$$\textbf{softmax}(z) = \left(\frac{\exp z_1}{Z}, \ldots, \frac{\exp z_K}{Z}\right)$$

where $Z = \sum_{k=1}^{K} \exp z_k$

Numerically Stable Softmax

- recall the definition of softmax for a vector $z \in \mathbb{R}^K$ is

$$\textbf{softmax}(z) = \left(\frac{\exp z_1}{Z}, \ldots, \frac{\exp z_K}{Z}\right)$$

  where $Z = \sum_{k=1}^{K} \exp z_k$

- **problem**: susceptible to overflow and underflow

Numerically Stable Softmax

- recall the definition of softmax for a vector $z \in \mathbb{R}^K$ is

$$\textbf{softmax}(z) = \left(\frac{\exp z_1}{Z}, \ldots, \frac{\exp z_K}{Z}\right)$$

  where $Z = \sum_{k=1}^{K} \exp z_k$

- **problem**: susceptible to overflow and underflow

- **solution**: let $z_{\text{max}} = \max\{z_1, \ldots, z_K\}$; then

$$\textbf{softmax}(z) = \left(\frac{\exp(z_1 - z_{\text{max}})}{Z}, \ldots, \frac{\exp(z_K - z_{\text{max}})}{Z}\right)$$

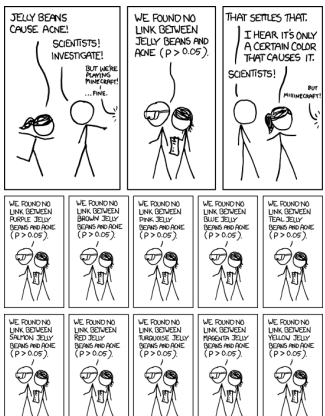
  where $Z = \sum_{k=1}^{K} \exp(z_k - z_{\text{max}})$

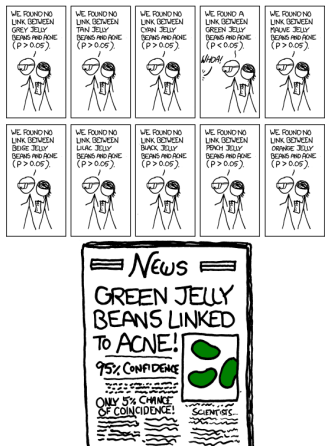
Algorithmic Issues

- poor experiment design (e.g., choosing a favourable random seed)
- systematic error from mini-batch sampling or division into train-test split
- leaking information from test set during training

Misinterpreting Statistical Experiments

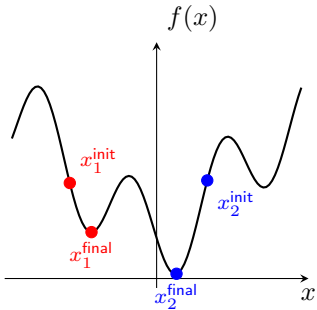
(https://m.xkcd.com/882/)





Favourable Random Seeds

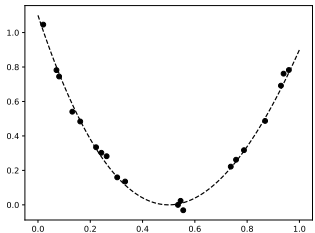
"random chance seems to have operated in our favour"



- gradient descent will take point to its closest local minimum†
- a lucky random seed can favour/penalize one model over another
- always repeat experiments initializing with different random seeds
  - compare different variants of model with the same random seeds

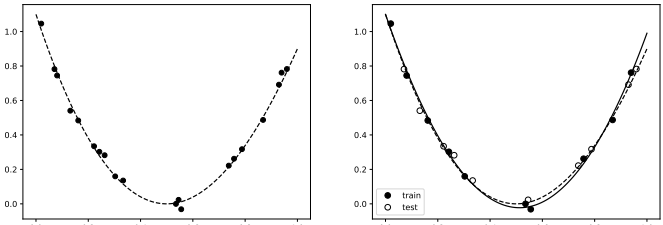
†so long as the function is sufficiently smooth

Sampling Strategies on Regression Example



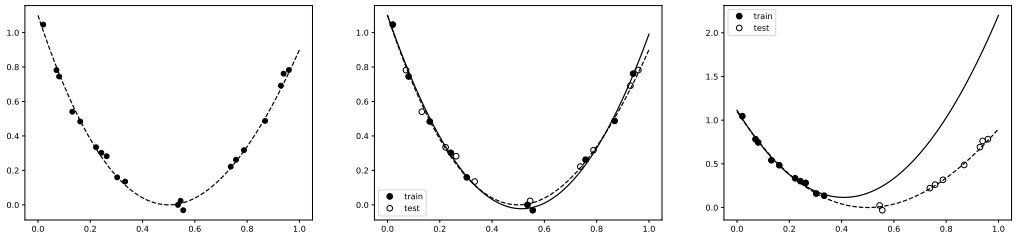
- training sample distribution should match test sample distribution
  - but avoid correlations between train and test splits, e.g., alternate video frames
- similar effect with stochastic gradient descent if no shuffling between epochs

Sampling Strategies on Regression Example



- training sample distribution should match test sample distribution
  - but avoid correlations between train and test splits, e.g., alternate video frames
- similar effect with stochastic gradient descent if no shuffling between epochs

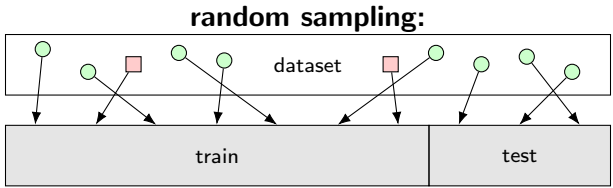
Sampling Strategies on Regression Example



- training sample distribution should match test sample distribution
  - but avoid correlations between train and test splits, e.g., alternate video frames
- similar effect with stochastic gradient descent if no shuffling between epochs

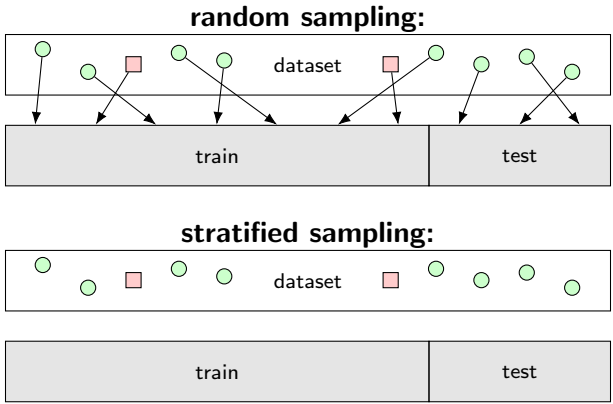
Unbalanced Data

- rare categories may have different distributions between train and test splits



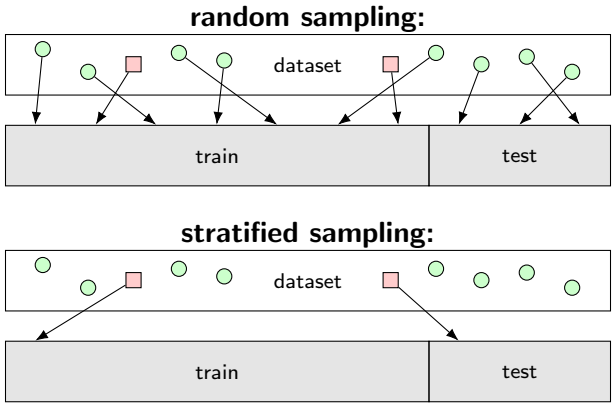
Unbalanced Data

- rare categories may have different distributions between train and test splits



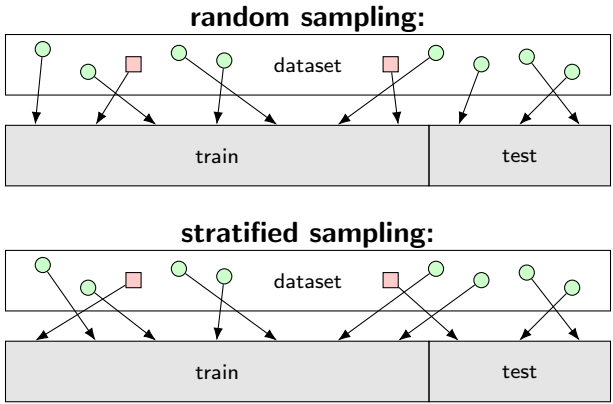
Unbalanced Data

- rare categories may have different distributions between train and test splits



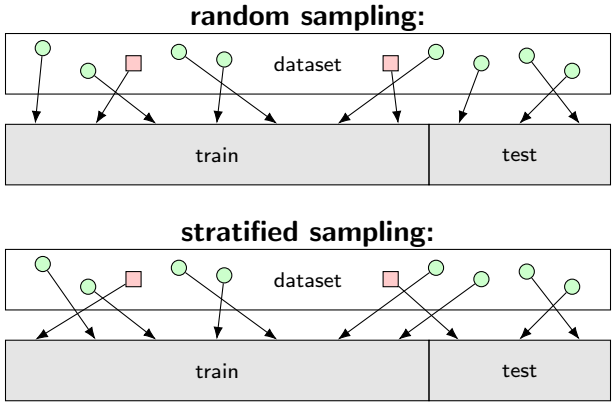
Unbalanced Data

- rare categories may have different distributions between train and test splits



Unbalanced Data

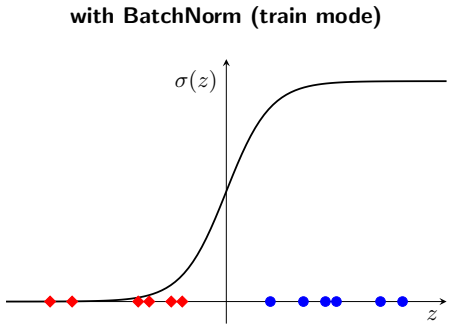
- rare categories may have different distributions between train and test splits



- can also be done in mini-batch sampling
- can also reweight rare samples in loss function

Information Leakage

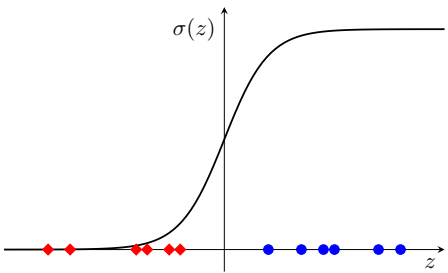
- test set data must never be touched during training/developing the model
- a common source of information leakage is when pre-processing the data
- another common error is not putting layers into evaluation mode, e.g., `BatchNorm`



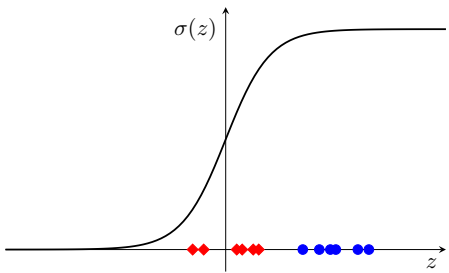
Information Leakage

- test set data must never be touched during training/developing the model
- a common source of information leakage is when pre-processing the data
- another common error is not putting layers into evaluation mode, e.g., `BatchNorm`

with BatchNorm (train mode)



no BatchNorm (eval mode)



Bugs in Machine Learning Algorithms can be Hard to Find

- consider trying to minimize the following scalar-valued function

$$f(x) = 10x^4 + x^2$$

- we'll use damped Newton's method, which has updates

$$x \leftarrow x - \eta \frac{f'(x)}{f''(x)}$$

Bugs in Machine Learning Algorithms can be Hard to Find

- consider trying to minimize the following scalar-valued function

$$f(x) = 10x^4 + x^2$$

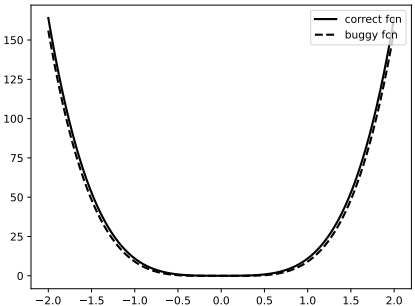
- we'll use damped Newton's method, which has updates

$$x \leftarrow x - \eta \frac{f'(x)}{f''(x)}$$

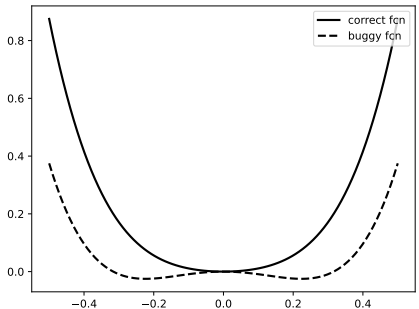
- let's introduce a small bug

correct	**buggy**
$f'(x) = 40x^3 + 2x$	$f'(x) = 40x^3 - 2x$
$f''(x) = 120x^2 + 2$	$f''(x) = 120x^2 - 2$

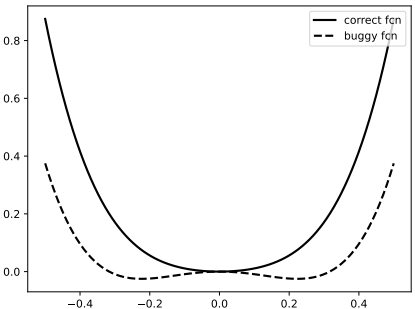
Bugs in Machine Learning Algorithms can be Hard to Find

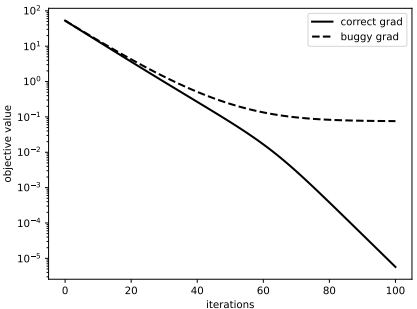


Bugs in Machine Learning Algorithms can be Hard to Find

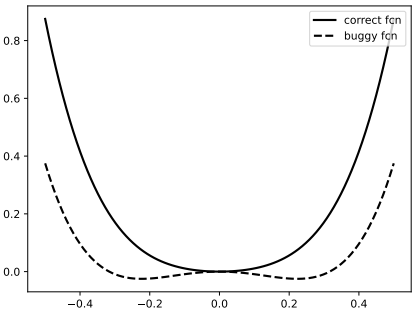


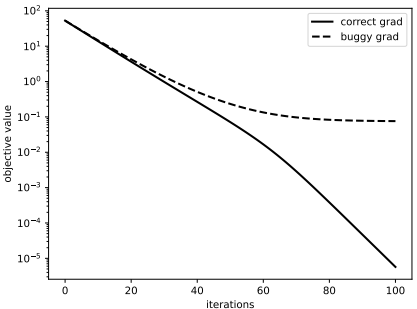
Bugs in Machine Learning Algorithms can be Hard to Find





Bugs in Machine Learning Algorithms can be Hard to Find



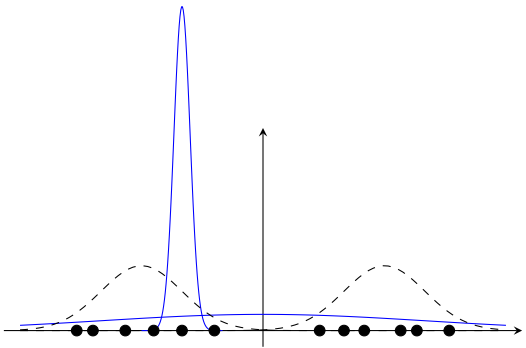


- **take home message:** use automatic differentiation wherever possible

Be Aware of Pathological Cases

- e.g., the true optimal solution of a Guassian mixture model is not desired

$$\text{maximize} \quad \sum_{i=1}^{N} \log \underbrace{\sum_{k=1}^{K} \pi_k \mathcal{N}(x^{(i)}; \mu_k, \Sigma_k)}_{p(x^{(i)};\theta)}$$



Implementation Tips

- **software is written for people, not machines**
- check for `NaN` and `Inf`
- assert pre- and post-conditions
- write simple test cases when debugging (or better yet, before you need to debug)
  - and keep them for future regression testing
- use existing code where possible
  - **deep learning frameworks:** PyTorch, TensorFlow, JAX
  - but don't blindly trust third-party code
  - reproduce published results before extending
- use software revision control (e.g., github)
  - do not store files than can be reproduced or downloaded (but include scripts)
- print out and save debugging information (but not within tight loops)
  - checkpoint models
- use `top`, Task Manager, or `nvidia-smi` to check for memory leaks
- run on small examples before your entire dataset

In case of fire

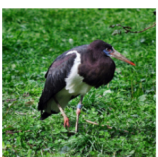
1. git commit
2. git push
3. leave building

Dataset Bias

- **every** dataset is biased!

Noisy/Erroneous Labels

- (almost) **every** dataset is wrong!

MNIST	CIFAR-10	CIFAR-100	Caltech-256	ImageNet	QuickDraw
					
given: 5 corrected: 3	given: cat corrected: frog	given: lobster corrected: crab	given: ewer corrected: teapot	given: white stork corrected: black stork	given: tiger corrected: eye

(source: `https://github.com/cleanlab/cleanlab`)

Diagnostic Tips

"experimental confirmation of a prediction is merely a measurement; experimental disproving of a prediction is a discovery"

- measure everything
- visualize everything
- choice of metrics can influence interpretation of results
  - ask questions of your metrics
- maintain a mental model of your code/algorithm
  - exercise your mental model against your observations
- develop diagnostic tests (next few slides)
- change one thing at a time, i.e., controlled experiments
  - e.g., different components of your loss function
- make sure learnable parameters (weights) are changing
  - make sure weights make a difference—freeze everything else and retrain from initialised weights or perturb during training
- **report trends not single results**
  - this is important not just for diagnoses but also scientific integrity

Loss Functions

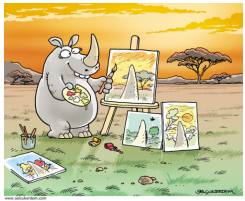
- loss functions are often constructed by summing many terms

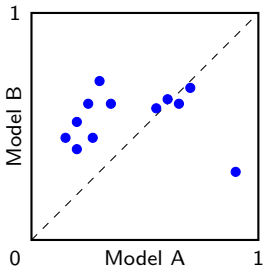
$$L(\theta) = \lambda_1 L_1(\theta) + \lambda_2 L_2(\theta) + \dots$$

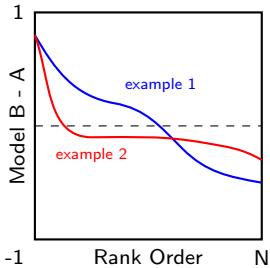
- investigate the contribution of each term by varying the weight of just that term
  - setting the weight to zero should remove the behaviour encouraged by that term
  - setting the weight very high should make that term dominate
- don't just tune hyper-parameters to get best performance, also tune to get better understanding
- not all loss functions have the same scale or are bounded
  - take this into account when setting the weight for each term

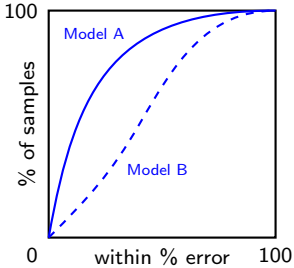
Visualize Your Data

- for high-dimensional features use t-SNE or PCA plots
- to discover correlations or compare models visualize different statistics



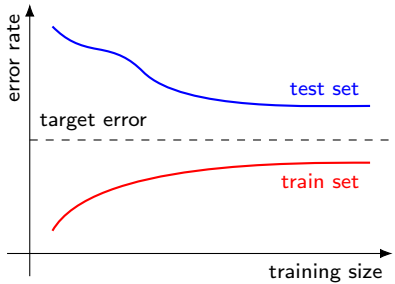




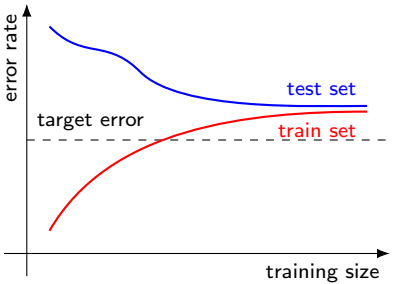


Diagnostic Example: Over-fitting vs. Poor Features

- suppose that our test error is unacceptably high and we suspect the problem is either that the model is **over-fitting** or the **features** are not good enough
  - first hypothesis suggests that training error will be much lower than test error
  - second hypothesis suggests that training error and test error will both be high

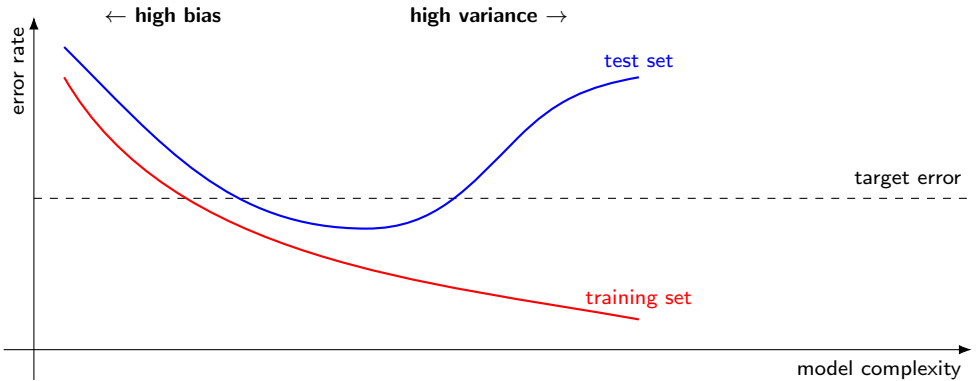


high variance

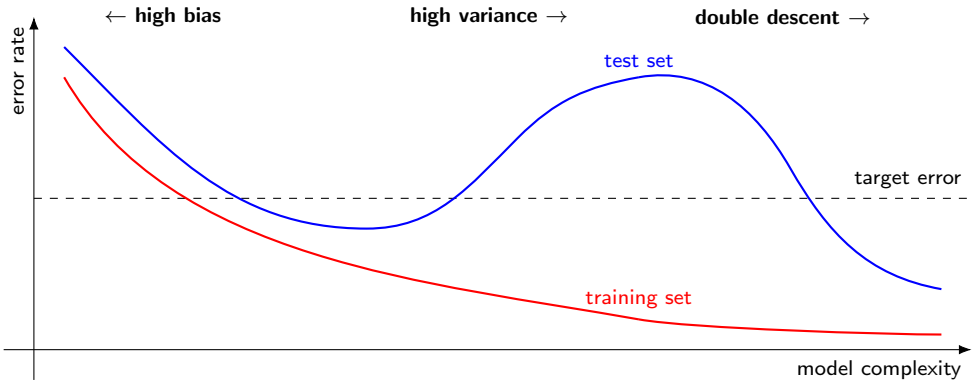


high bias

Bias/Variance Trade-off



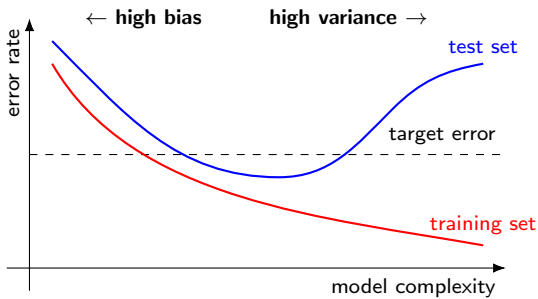
Bias/Variance Trade-off



- **double descent** has been observed in deep learning, where further increasing model complexity sometimes results in test set error dropping again
  - not well understood theoretically

Fixes for Bias/Variance Problems

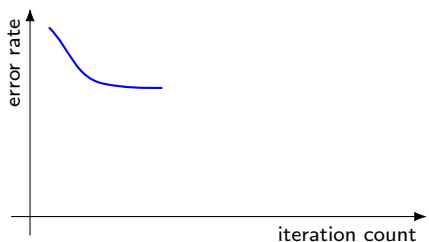
- diagnosing bias and variance problems provides hints as to what to try next
- **for bias problems**:
  - try a larger set of features
  - try a richer model class
- **for variance problems**:
  - try getting more training examples
  - try data augmentation
  - try fewer features
  - try training for longer (and hope for double descent)



Diagnostic Example: Objective vs. Optimization Algorithm

- we may suspect that our poor performance is due to either a problem with our **optimisation algorithm** (e.g., not running for long enough) or a problem with our **objective** (i.e., loss function)

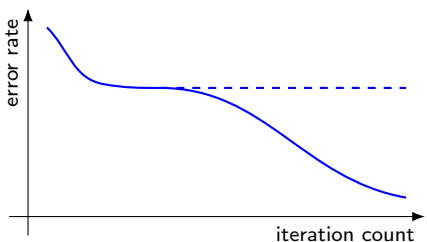
- unfortunately, it is often very difficult to determine whether training has converged



Diagnostic Example: Objective vs. Optimization Algorithm

- we may suspect that our poor performance is due to either a problem with our **optimisation algorithm** (e.g., not running for long enough) or a problem with our **objective** (i.e., loss function)

- unfortunately, it is often very difficult to determine whether training has converged



Diagnosing Optimization Issues

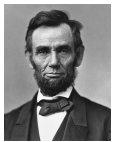
- suppose we care about maximizing some accuracy measure, $J(\theta)$, and our learning algorithm is trying to minimize a surrogate loss, $L(\theta)$
  - let $\theta^\star$ be the parameters returned by the learning algorithm
  - let $\theta^\dagger$ be any other parameters (e.g., guessed or from a different learning algorithm)

	$J(\theta^\dagger) > J(\theta^\star)$	$J(\theta^\dagger) < J(\theta^\star)$
$L(\theta^\star) < L(\theta^\dagger)$	wrong loss	no problem (?)
$L(\theta^\star) > L(\theta^\dagger)$	poor optimization	poor optimization (but got lucky)

Fixes for Optimization/Objective Problems

- diagnosing optimization versus objective problems provides us with hints as to what to try next
- **for optimization problems**:
  - try running for more iterations
  - try using a different algorithm (e.g., AdamW instead of SGD)
  - try random restarts
  - try smoothing (e.g., momentum, exponential moving average)
- **for objective problems**:
  - try different regularization
  - try data augmentation
  - try weighting training examples
  - try a different loss function
  - change the model

Diagnostic Summary



"give me six hours to chop down a tree and I will spend the first four sharpening my axe"

- diagnostics are an important tool when developing your machine learning method
  - we showed examples for bias/variance and optimization/objective but there are many others
  - diagnostics can save a lot of wasted effort by guiding your choice of what to try next
  - they also allow you to develop insights into your particular application and justify your design decisions
- diagnostics often involve repeated experiments with different parameter settings while keeping everything else fixed
  - be organized
- another important diagnostic tool is that of **error analysis**, i.e., understanding where your method is making mistakes, and **ablation analysis**, i.e., understanding which parts of your model help the most

Error Analysis and Ablation Analysis

Error Analysis

- tries to explain the difference between **current** and **perfect** performance
- how much error is due to different model components?
  - plug ground-truth (if available) into each component and see how it affects accuracy
  - add noise to each component and see how it affects accuracy
- does the algorithm fail on a particular subclass of examples?
  - visualize the data and results!

Ablation Analysis

- tries to explain the difference between some **baseline** and **current** performance
- often methods are built in an ad hoc fashion
  - features/components are added in early development and kept even if not needed
- remove features/components from the model one at a time and sees which results in the biggest decrease in performance
  - note that the order of removal matters
- allows simplification of the model, which can then be further developed

Advice for Getting Unstuck

"if you can't solve a problem, then there is an easier problem that you [also] can't solve: find it"

- find a simpler (related) problem
- solve subproblems
- explore boundary conditions
- read papers (both old and new) for ideas
  - be skeptical about results reported in the literature that are not reproducible
- talk to friends and colleagues
  - rubber duck debugging
- introduce more information and slowly remove
  - extreme case: "cheat" by introducing ground-truth features
  - can also test on random features — how should your model perform?
  - generate synthetic test cases

weights & biases

Weights & Biases (wandb)

Weights & Biases (wandb) is the de facto standard tool for:

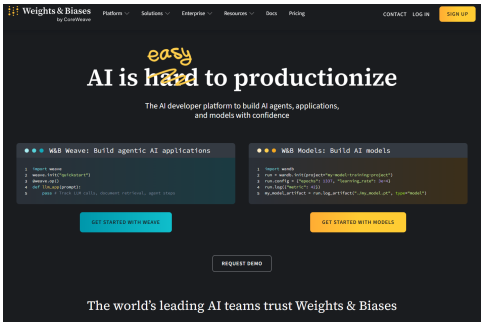
- monitoring training of deep learning models
- streamlining hyperparameter tuning
- logging and recording experiments
- visualizing metrics and internal model features

Requires:

- account on `wandb` server
- local Python package(s)

Not strictly needed for your projects but useful if you're running lots of experiments.

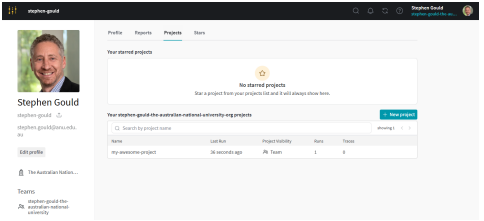
Quick Start

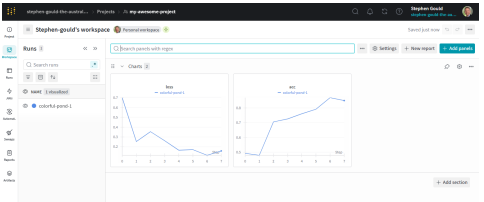


- create (free) account on website
  - `https://wandb.ai`
  - choose academic or personal use
  - choose "W&B Models"
  - choose "Python / PyTorch"
  - generate (and save) API key
- install `wandb` API
  - `pip install wandb`
- cache login credentials
  - `wandb login`
  - paste saved API key

Toy Example: Logging Metrics

```python
from random import random
import wandb

# start a new wandb instance
run = wandb.init(
    entity="...",
    project="my-awesome-project",
    name="first run",
)

# simulate training run
epochs = 10
offset = random() / 5
for epoch in range(2, epochs):
    acc = 1 - 2**-epoch - random() / epoch - offset
    loss = 2**-epoch + random() / epoch + offset

    # log metrics to wandb
    run.log({"acc": acc, "loss": loss})

# finish the run and upload any remaining data
run.finish()
```

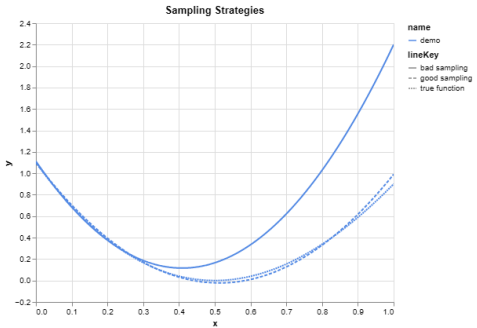




- running multiple times adds additional curves to each plot

Wandb Custom Charts

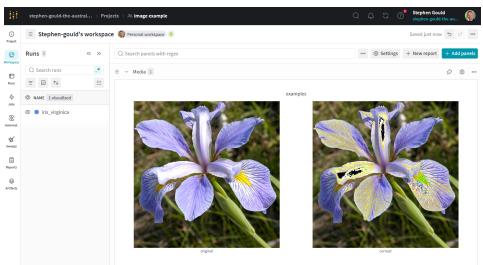
```python
# initialize data
x = np.linspace(0.0, 1.0, 100)
y_true = ...
y_good = ...
y_bad = ...

# log to wandb
wandb.init(project="regression example", name="demo")
chart = wandb.plot.line_series(
    xs=x.T,
    ys=np.vstack((y_true, y_good, y_bad)),
    keys=["true", "good", "bad"],
    title="Sampling Strategies")
wandb.log({"regression_curves": chart})
wandb.finish()
```



- other charts include `confusion_matrix`, `pr_curve`, `scatter`, `bar`

Wandb Images

```python
# load image
img = cv2.imread(BASENAME + ".jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# add one to each pixel
img2 = img + 1

# start wandb run and log images
wandb.init(project="image example",
    name="iris_virginica")
with wandb.init() as run:
    run.log({"examples": [
        wandb.Image(img, caption="original"),
        wandb.Image(img2, caption="corrupt"),
        ]})
```



Wandb Checkpointing

- checkpoints allow you to find your best performing model after training
- resume training from some earlier epoch with different setup
  - **danger:** this can make reproducing your results very difficult if you're not careful to document exactly what you've done (e.g., new learning rate, data augmentation, etc)

```
with wandb.init(...) as run:
    # create a model checkpoint
    artifact = wandb.Artifact(name="my_awesome_model", type="model")

    # add model weights to the artifact
    artifact.add_file("model.pt")

    # log the artifact to wandb
    run.log_artifact(artifact, alias=[f"epoch:{epoch}", f"acc:{accuracy}", "best"])
```

```
# resume training from a earlier checkpoint
with wandb.init(...) as run:
    artifact = run.use_artifact("my_awesome_model:best")
    artifact.download()
```

Wandb Hyperparameter Search

- W&B provides a systematic way to try different combinations of hyperparameters and keep track of the results
- involves defining a **sweep** specifying search strategy and metric

```
# configure the sweep
config = {'method': 'grid'}

config['metric'] = {
    'name': 'loss',
    'goal': 'minimize'
}

config['parameters'] = {
    'optimizer' : {'values': ['adam', 'sgd']},
    'fc_layer_size': {'values': [128, 256, 512]},
    'learning_rate': {'distribution': 'uniform', 'min': 1.0e-6, 'max': 1.0e-2},
    'batch_size': {'values': [32, 64, 128]}
}
```

- write your training script to use values from `run.config` (one set of options)
- start the sweep to automate calling training script with different options
  - can be done on multiple machines and repeated multiple times
- see the docs for lots more features and examples